

Confidence-Aware CNN

Software Design and Handover

Repository: github.com/003shrey/Confidence-Aware-CNN

Document Date: 27/09/2026

Reference Revision: 8c50524

Document Guide

Section	Page
Abstract, Project Caveats	1
1. Purpose, 2. System Overview, 3. Repository Components	2
4. How the Current System Works, 5. How to Make a Change, 6. Known Limitations	3
7. Handoff, 8. Implementation Contract, Results Snapshot	4
Current System Workflow, Future Development	5
Future End-to-End Workflow, Deployment and Monitoring	6

Abstract

This project explores a confidence-aware CNN for MNIST digit classification. The system combines prediction confidence, temperature scaling, and threshold-based abstention to determine when to provide a predicted digit and when to return "I don't know." The experiment evaluates confidence calibration, threshold behavior, and confidence on unfamiliar inputs, while recording inference decisions for analysis.

Project Caveats

Notebook-Driven Implementation

The current experiment is implemented in Jupyter notebooks, with `notebooks/day_5_full_evaluation.ipynb` serving as the consolidated execution path. There are no separate production Python modules, APIs, services, or databases.

Saved Model Artifacts

The trained CNN and fitted calibration parameters are stored under `artifacts/` and can be loaded for inference without retraining.

Experimental Boundary

The CIFAR-10 evaluation is a limited unfamiliar-input comparison and does not establish a general solution for detecting every possible unfamiliar input.

1. Purpose

The purpose of this project is to investigate whether a CNN-based digit classifier can use calibrated confidence to decide when to provide a prediction and when to abstain. The experiment evaluates confidence calibration, threshold-based decision making, behavior on unfamiliar inputs, and structured logging of inference decisions.

2. System Overview

Step	Stage	What happens
1	Prepare images	MNIST images are scaled to 0-1 and given a grayscale channel dimension.
2	Train the CNN	A small Keras CNN classifies the ten MNIST digits, with 10% of training data held out for validation.
3	Measure confidence	The highest softmax probability is treated as the model's confidence for its predicted digit.
4	Calibrate confidence	Temperature scaling learns one temperature on MNIST validation data and applies it to the model logits before softmax.
5	Check unfamiliar inputs	A seeded CIFAR-10 subset is converted to grayscale and resized to 28 x 28 so the MNIST model can process it. CIFAR-10 labels are not used to score digit correctness.
6	Analyze thresholds	Threshold sweeps show how coverage, abstention, and MNIST selective accuracy change as the confidence cutoff changes.
7	Make and log a decision	The inference helper returns the predicted digit when confidence meets the cutoff; otherwise it returns "I don't know" and records the decision as JSON.

3. Repository Components

Component	Location	Purpose
Baseline and exploration	notebooks/day2- exploration.ipynb	Earlier model exploration
Clean baseline	notebooks/day2-Baseline_model.ipynb	Clean MNIST baseline
Calibration	notebooks/day 3 temp_calibration.ipynb	Temperature scaling and confidence comparison
Failure analysis / logging	notebooks/day_4 failure_analysis&logging.ipynb;	Threshold analysis, failures, and logging helper
Full evaluation	notebooks/day_5_full_evaluation.ipynb	Consolidated experiment and full-scale logging evaluation
Sample records	logs/day4_inference.jsonl	Three example decisions
Full evaluation log	logs/day5_evaluation.jsonl	Persisted evaluation records
Readable preview	logs/day5_evaluation_preview_100.md	Human-readable preview of evaluation records
Project scope	README.md / REQUIREMENT.md	Purpose, scope, and goals
Decision history	DECISIONS.md	Important choices and reasoning
Dependencies	requirements.txt	Python packages

The experiment logic is implemented in notebooks. There are no separate production Python modules, APIs, services, or databases.

4. How the Current System Works

Input preprocessing

MNIST images are divided by 255 and reshaped to 28 x 28 x 1. CIFAR-10 images are divided by 255, converted from RGB to grayscale, and resized to 28 x 28. This makes the CIFAR-10 images acceptable as inputs to the MNIST model, but does not turn them into digit data.

CNN prediction and confidence

The baseline is a Keras Sequential CNN with two convolution/pooling stages, a 64-unit dense layer, and a ten-class softmax output. The predicted digit is the class with the highest probability. That probability is used as confidence.

Temperature scaling

The calibration step reconstructs the model logits, learns one temperature using MNIST validation logits and labels, and applies logits divided by that temperature before softmax. The MNIST test set is used for comparison, not for fitting the temperature. CIFAR-10 is used only for the confidence comparison.

Threshold decision

The inference helper compares the maximum probability with the supplied threshold. At or above the threshold it returns the predicted digit and an "answer" verdict. Below the threshold it returns "I don't know" and no prediction. The diagnostic threshold is 0.95. The wider 0.50-0.95 sweep is used to observe the coverage/abstention trade-off and does not select a new cutoff.

Structured logging

Each logged decision contains request_id, dataset, prediction, confidence, threshold, and verdict. The repository contains a small Day 4 sample log and a larger Day 5 evaluation log, plus a 100-record readable preview.

5. How to Make a Change

Change	Where to look
Change MNIST preprocessing or the CNN	Start in the setup and baseline sections of day_5_full_evaluation.ipynb. Use day2-Baseline_model.ipynb as a reference.
Change CIFAR-10 preprocessing or subset size	Edit the CIFAR-10 section of day_5_full_evaluation.ipynb.
Change temperature fitting or ECE	Edit the calibration section of day_5_full_evaluation.ipynb; the separate Day 3 notebook contains the original calibration work.
Change the threshold sweep	Edit thresholds and sweep_confidence_thresholds in the Day 4 portion of day_5_full_evaluation.ipynb.
Change answer/abstain behavior or log fields	Edit predict_with_abstention in the Day 4 portion, then rerun the logging checks and full evaluation.
Inspect high-confidence failures	Use the MNIST and CIFAR-10 failure-analysis sections of day_5_full_evaluation.ipynb.
Rerun the full evaluation	Run day_5_full_evaluation.ipynb from top to bottom to retrain and regenerate the evaluation. Saved model and calibration artifacts can be loaded for inference without retraining.
Inspect persisted evidence	Review logs/day4_inference.jsonl for the sample decisions and logs/day5_evaluation.jsonl for the full evaluation.

6. Known Limitations

- CIFAR-10 is a small, clearly different comparison. It does not establish behavior on unfamiliar images that resemble handwriting.
- CIFAR-10 predictions are digit outputs from the MNIST model, not CIFAR-10 classifications, so the experiment does not score them for correctness.
- The model, calibration helpers, threshold logic, and logger are notebook code rather than an importable application layer.
- No automated pytest suite is present.
- The repository contains two JSONL inference logs from different stages of the experiment: logs/day4_inference.jsonl contains the initial three inference tests, while logs/day5_evaluation.jsonl contains the full evaluation run. They are intentionally kept as separate evidence.
- The repository does not provide a general solution for detecting every possible unfamiliar input.

7. Handoff

- Read REQUIREMENT.md for the problem and scope, DECISIONS.md for the reasoning behind the experiment, and this document for the current system flow and change points.
- Before changing behavior, run the consolidated Day 5 notebook in order so the model, logits, probabilities, and evaluation state are created consistently.
- Treat the notebooks as the source of executable experiment logic and the committed logs as evidence from completed runs.
- When changing logging or inference behavior, check the configured output path and update the persisted evidence path consistently.
- The project is intentionally an experiment rather than a production service: there are no APIs, databases, deployment setup, or production modules.

Reproduction: Run notebooks/day_5_full_evaluation.ipynb from top to bottom after installing the dependencies listed in requirements.txt. The notebook can retrain the model and regenerate the evaluation; saved model and calibration artifacts are available for inference without retraining

Document boundary: this handover describes the current repository design. Experiment results and decision reasoning remain in the notebooks and DECISIONS.md rather than being duplicated here.

8. Implementation Contract

The following values are the current implementation details needed to reproduce the consolidated experiment without inferring them from the notebooks.

Reproducibility	TensorFlow/Keras; seed = 42; deterministic TensorFlow operations enabled.
MNIST	60,000 training images; 10,000 test images; divide by 255; reshape to 28×28×1; validation_split = 0.1.
CNN	Input (28,28,1) → Conv2D(32, 3×3, ReLU) → MaxPool(2) → Conv2D(64, 3×3, ReLU) → MaxPool(2) → Flatten → Dense(64, ReLU) → Dense(10, softmax).
Training	Adam; sparse categorical cross-entropy; batch size 128; 5 epochs; shuffle=True.
Confidence	prediction = argmax(probabilities); confidence = max(probabilities).
ECE	15 equal-width bins from 0.0 to 1.0; weighted absolute difference between bin accuracy and mean confidence.
Calibration	Temperature scaling; optimize log(T) with Adam(lr=0.01) for 200 steps on MNIST validation logits/labels; T = exp(log(T)).
CIFAR-10	1,000 test images sampled without replacement using NumPy default_rng(42); /255 → RGB-to-grayscale → resize 28×28. Labels are not used for correctness.
Decision	Diagnostic cutoff = 0.95. confidence ≥ cutoff → predicted digit + "answer"; otherwise prediction = null + "I don't know".
Sweep	Thresholds = 0.50, 0.55, ..., 0.95. Diagnostic only; no new cutoff is selected.
Logging	JSONL fields: request_id, dataset, prediction, confidence, threshold, verdict. Day 4 inference helper writes the three sample decisions to logs/day4_inference.jsonl. The full Day 5 evaluation is persisted separately in logs/day5_evaluation.jsonl.
Rerun	Run notebooks/day_5_full_evaluation.ipynb top-to-bottom to retrain and regenerate the evaluation. Saved artifacts can be loaded for inference without retraining.
Saved Model	artifacts/mnist_confidence_model.keras - trained Keras CNN & artifacts/calibration.json - fitted temperature and inference-time configuration.

Reconstruction check

- A new engineer should be able to reproduce the data preparation, CNN training, calibration, threshold sweep, and logging contract from this page.
- For behavior changes, the consolidated Day 5 notebook remains the executable source; this section records the values that should not be inferred.

Results Snapshot

Record the original consolidated-run results here so a new run can be sanity-checked against the documented experiment.

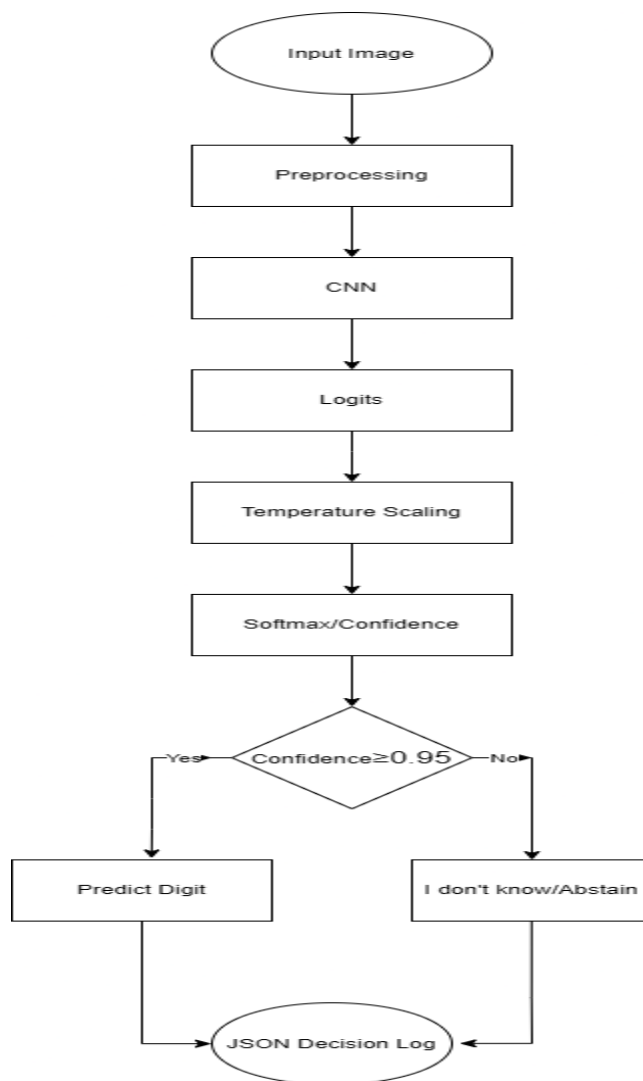
Metric	Original run
MNIST test accuracy	98.93%
Learned temperature (T)	1.1658
ECE before calibration	0.0023
ECE after calibration	0.0025
0.95 coverage	95.01%
0.95 abstention rate	4.99%
0.95 selective accuracy	99.86%

Further Development

The current repository is an experiment rather than a production application. A natural next step is to turn the validated experiment into a complete end-to-end system while keeping the notebooks as evidence of how the behavior was evaluated.

Current Model Workflow

This is the inference path represented by the current design. Temperature scaling is fitted earlier on validation data; during inference it is applied to the model logits before the final confidence value is used for the decision.



1. Extract the Experiment into Application Code

Move the working logic out of the notebooks into small, reusable Python modules for image preprocessing, CNN inference, confidence calibration, threshold-based abstention, and structured logging. The notebooks should remain as experiment evidence rather than the main execution path.

2. Load the Saved Model and Calibration Artifacts

Load the committed Keras model and fitted calibration parameters for inference without retraining. The saved artifacts should contain enough information to reproduce prediction and confidence behavior consistently.

3. Add an End-to-End Inference Interface

Create a simple interface where a user provides an image and receives the predicted digit when confidence is above the threshold, or "I don't know" when confidence is below it. The response should also expose the confidence and record the decision in the structured log.

4. Add Application-Level Testing

Add tests for the decision logic and important preprocessing behavior, including confidence above and below the threshold, expected input shape, and the required logging fields.

Further Development

Extending the current experiment into an end-to-end system

The following items are proposed extensions and are not part of the current reference implementation.

Future End-to-End Workflow

The deployment path can extend the current inference flow without changing the core decision logic:

User Image	submitted to the application
Application / API	receives the request
Preprocessing	formats the image
Saved CNN	generates logits / probabilities
Calibration	applies the saved temperature
Threshold	answers or abstains
Response	digit + confidence, or "I don't know"
Logging / Monitoring	records the decision for analysis

5. Package and Deploy

Package the application so it can run outside the development environment and deploy it as a complete service. The exact hosting platform, authentication, monitoring, and scaling requirements should be decided based on the intended use rather than added prematurely.

6. Monitor and Re-evaluate

After deployment, use structured inference data to monitor abstention rate, confidence distribution, and high-confidence failures. New evaluation data should be used to decide whether the current threshold and confidence behavior still match the intended use.

This development path keeps the existing experiment small while creating a clear route from validated notebook behavior to a reusable, deployable inference system.